SHERLOCK

Security Review For Lombard Finance



Collaborative Audit Prepared For: **Lombard Finance**
Lead Security Expert(s): **Bauchibred**
defsec
Date Audited: **June 25 - June 26, 2026**

Introduction

This audit covers the contract changes that introduce per-minter mint rate limiting across the Lombard tokens. A new RateLimitsV2 library provides a gas-optimized single-slot leaky-bucket limiter that replaces the legacy RateLimits, and BaseLBTC wires it into the mint flow so that every mint consumes a per-minter bucket keyed by address and configured by the manager role. NativeLBTC, StakedLBTC, and BridgeTokenAdapter route their mint entrypoints – both privileged and permissionless proof-mint paths – through this limiter, with StakedLBTC drawing all mints from a shared AssetRouter bucket. The new state is appended to the existing storage layouts for in-place proxy upgrades.

Scope

Repository: lombard-finance/smart-contracts

Audited Commit: fcce2a8eae335a79aad44c977edf7bbeeac7ce29

Final Commit: 420034de2a9dfc35646d46b361cfc27007649f56

Files:

- contracts/LBTC/BaseLBTC.sol
- contracts/LBTC/BridgeTokenAdapter.sol
- contracts/LBTC/NativeLBTC.sol
- contracts/LBTC/StakedLBTC.sol
- contracts/libs/RateLimitsV2.sol

Final Commit Hash

420034de2a9dfc35646d46b361cfc27007649f56

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	1	9

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: StakedLBTC release claims share the AssetRouter fee bucket [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/issues/11

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Normal `StakedLBTC` release mints consume the same `AssetRouter` minter bucket as redeem fee and `mintWithFee` traffic, so one user's fee activity can spend shared quota and block another user's inbound release claim, and inbound release traffic can also block fee-bearing flows.

Vulnerability Details

The new `AssetRouter` docs hint that
redeem fee mints and mintWithFee double counting against the router bucket.

```
/// @dev Fee collection on redeem and StakedLBTC deposit paths uses burn-then-mint
 ↪  to treasury
///      (not a user transfer) so callers do not need a separate fee approval.
 ↪  Those treasury
///      mints go through the token's `MINTER_ROLE` entrypoint and consume the
 ↪  {AssetRouter}
///      minter rate-limit bucket; burns do not release bucket capacity.
```

But ordinary release claims also end at a
token side mint() from AssetRouter.handlePayload().

```
function handlePayload(
    GMPUtils.Payload memory payload
) external override returns (bytes memory) {
    ...
    IBaseLBTC(receipt.toToken).mint(receipt.recipient, receipt.amount);
}
```

So for `StakedLBTC`, we reach the external mint(address,uint256) entrypoint, where
the limiter is keyed by msg.sender.

```
function mint(
    address to,
    uint256 amount
) external override onlyRole(MINTER_ROLE) whenMintBurnAllowed {
    _mintWithLimit(to, amount);
```

```
}

function _consumeMinterLimit(address minter, uint256 amount) internal {
    RateLimitsV2.updateLimit(
        _getBaseLBTCStorage().minterRateLimits[minter],
        amount
    );
}
```

Since `msg.sender` in this case is the `AssetRouter`, release claims share the same bucket already used by redeem fee mints and also the fee leg of mintWithFee, i.e a user's redeem fee can consume the shared router quota and make Bob's unrelated inbound `StakedLBTC` release claim revert once the bucket is exhausted.

Note that `NativeLBTC` and `BridgeTokenAdapter` do not couple proof mints this way because their proof paths consume `address(this)` instead.

Impact

One user's redeem-fee or `mintWithFee` traffic can consume the shared `AssetRouter` quota and block another user's unrelated inbound `StakedLBTC` release claim also inbound release traffic can also consume the same quota and block fee-bearing flows.

Recommended Mitigation Steps

Route ordinary `StakedLBTC` release mints through a dedicated mint identity or dedicated rate-limit bucket, instead of sharing the `AssetRouter` bucket with redeem-fee and `mintWithFee` traffic.

Discussion

hashxtree

As we discussed on the call, we don't want to touch `AssetRouter` right now. The fix is to use approvals instead of fee remint. Are there enough docs to mark it as acknowledged?

Bauchibred

No, I think we can add some more details at AssetRouter.sol#L21-L43 in regards to this.

hashxtree

https://github.com/lombard-finance/smart-contracts/pull/524

Bauchibred

Acknowledged.

Issue L-1: Stale `checkSignature` NatSpec after EIP-7702 change [RESOLVED]

Source: https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/issues/6

Summary

`EIP1271SignatureUtils.checkSignature` was updated so that `isEoaAccount` treats EIP-7702 delegation-designator accounts (23-byte code `0xef0100||addr`) as EOAs and verifies them via `ECDSA.recover` instead of ERC-1271. The `checkSignature` NatSpec was not updated and still describes only the old "no code = EOA / has code = contract (ERC-1271)" split, so the documented behavior no longer matches the implementation.

Vulnerability Detail

`isEoaAccount` returns `true` for two cases – accounts with no code, and accounts whose code is exactly the 23-byte EIP-7702 delegation designator (`0xef`, `0x01`, `0x00` followed by the delegate address). `checkSignature` then routes every "EOA" through `ECDSA.recover`:

- No-code account → ECDSA
- EIP-7702 delegated account → ECDSA (new behavior)
- Any other contract code → ERC-1271 `isValidSignature`

The docstring on `checkSignature`, however, still states that any `signer` containing code is validated by passing the signature to the signer's ERC-1271 `isValidSignature`.

Impact

Documentation no longer reflects actual behavior.

Code Snippet

```
/**
 * @notice Checks @param signature is a valid signature of @param digest from
 ↪  @param signer.
 * If the `signer` contains no code -- i.e. it is not (yet, at least) a contract
 ↪  address, then checks using standard ECDSA logic
 * Otherwise, passes on the signature to the signer to verify the signature and
 ↪  checks that it returns the `EIP1271_MAGICVALUE`.
 */
function checkSignature(
    address signer,
```

```
    bytes32 digest,
    bytes memory signature
) internal view returns (bool) {
    if (!isEoaAccount(signer)) {
        if (
            IERC1271(signer).isValidSignature(digest, signature) !=
            EIP1271_MAGICVALUE
        ) {
            return false;
        }
    } else {
        if (ECDSA.recover(digest, signature) != signer) {
            return false;
        }
    }
    return true;
}

function isEoaAccount(address account) internal view returns (bool) {
    if (account.code.length == 0) {
        return true;
    }
    if (
        account.code.length == 23 &&
        account.code[0] == 0xef &&
        account.code[1] == 0x01 &&
        account.code[2] == 0x00
    ) {
        return true;
    }
    return false;
}
```

Tool Used

Manual Review

Recommendation

Update the `checkSignature` NatSpec to describe all three branches: no-code EOA → ECDSA; EIP-7702 delegated account → ECDSA against the underlying EOA key; other contract code → ERC-1271.

Discussion

555-andrew

Acknowledged. Fixed here:
https://github.com/lombard-finance/smart-contracts/pull/513

defsec

Fix is confirmed.

Issue L-2: `_mintWithFee` fee-bound inconsistency (`amount == fee` allowed) [RESOLVED]

Source: https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/issues/7

Summary

`AssetRouter._mintWithFee` rejects only `amount < fee`, allowing `amount == fee` (recipient nets zero), whereas `NativeLBTC._mintWithFee` uses the stricter `fee >= amount`. Inconsistent fee bounds across the two paths.

Vulnerability Detail

```
782:788:smart-contracts/contracts/LBTC/AssetRouter.sol
        if (amount < fee) {
            revert AssetRouter_FeeGreaterThanAmount();
        }
        if (fee > 0) {
            tokenContract.burn(recipient, fee);
            tokenContract.mint(treasury, fee);
        }
```

When `amount == fee`, the full minted amount is immediately burned and re-minted to treasury; the recipient receives nothing. `fee` is capped by `maximumMintCommission` and the user's signed approval, but the behavior differs from `NativeLBTC._mintWithFee` (`fee >= amount` reverts).

Impact

Edge-case inconsistency.

Code Snippet

https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/blob/main/smart-contracts/contracts/LBTC/NativeLBTC.sol#L424

Tool Used

Manual Review

Recommendation

Align both paths to the same bound (e.g. revert on `amount <= fee`).

Discussion

hashxtree

https://github.com/lombard-finance/smart-contracts/pull/522

Issue L-3: `AssetRouterChanged` event parameter order mismatch [RESOLVED]

Source: https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/issues/8

Summary

`IBaseLBTC` declares `AssetRouterChanged(address indexed newVal, address indexed prevVal)`, but all implementations emit `(prevValue, newVal)`. Runtime and tests are internally consistent, but any indexer/doc consumer reading the interface gets swapped `prev/new` semantics.

Vulnerability Detail

Implementations emit `(prev, new)`:

```
function _changeAssetRouter(address newVal) internal {
    NativeLBTCStorage storage $ = _getNativeLBTCStorage();
    address prevValue = address($.assetRouter);
    $.assetRouter = IAssetRouter(newVal);
    emit AssetRouterChanged(prevValue, newVal);
}
```

The interface declares the parameters in the opposite order (`newVal` first, `prevVal` second), so the indexed topic labels are reversed relative to the emitted values. `StakedLBTC._changeAssetRouter` and `BridgeTokenAdapter._changeAssetRouter` follow the same `(prev, new)` emit.

Impact

Informational.

Code Snippet

https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/blob/main/smart-contracts/contracts/LBTC/StakedLBTC.sol#L517

https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/blob/main/smart-contracts/contracts/LBTC/interfaces/IBaseLBTC.sol#L42

Tool Used

Manual Review

Recommendation

Change the interface declaration to `AssetRouterChanged(address indexed prevVal, address indexed newVal)` to match all emitters.

Discussion

hashxtree

https://github.com/lombard-finance/smart-contracts/pull/514

defsec

Fix is confirmed.

Issue L-4: Unused `Math` import in `BridgeTokenAdapter` and `StakedLBTC` [RESOLVED]

Source: https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/issues/9

Summary

`Math` (`@openzeppelin/contracts/utils/math/Math.sol`) is imported in `BridgeTokenAdapter.sol` and `StakedLBTC.sol` but never used in either contract.

Vulnerability Detail

Both files import `Math` but contain no `Math.` reference anywhere in the contract body:

For contrast, `NativeLBTC.sol` imports `Math` and actually uses it (`Math.min(maxFee, feeAction.fee)` at L422), so the import is justified there, but not in `BridgeTokenAdapter` or `StakedLBTC`.

Impact

An unused import has no runtime effect; it is dead code that slightly increases compilation surface and reduces readability.

Code Snippet

https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/blob/main/smart-contracts/contracts/LBTC/BridgeTokenAdapter.sol#L5

https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/blob/main/smart-contracts/contracts/LBTC/StakedLBTC.sol#L6

Tool Used

Manual Review

Recommendation

Remove the unused `import {Math} ...` statement from `BridgeTokenAdapter.sol` and `StakedLBTC.sol`.

Discussion

hashxtree

https://github.com/lombard-finance/smart-contracts/pull/515

defsec

Fix is confirmed.

Issue L-5: Redundant zero limit validation in `RateLimitsV2.setRateLimit` [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/issues/10

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`RateLimitsV2.setRateLimit()` checks `config.limit == 0 || config.window == 0`, but the current LBTC wrappers already intercept `config.limit == 0` and delete the mapping entry instead of calling the library. That makes the `config.limit == 0` half of the library revert unreachable from today's LBTC call graph, while `config.window == 0` remains the only live validation enforced there.

Vulnerability Details

In both BaseLBTC setMinterLimit() and BridgeTokenAdapter setMinterLimit(), a zero `config.limit` takes the delete branch and never reaches the config.limit == 0 || config.window == 0 revert inside RateLimitsV2.setRateLimit(). The duplicated zero-limit validation is not reachable, also only the `window == 0` half is actually protecting a live invariant since a stored zero window makes availableAmount() treat the bucket as fully decayed on every call

Impact

Informational

Code Snippet

https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/blob/45ee91f30c14011662326b05d9663d589189e529/smart-contracts/contracts/LBTC/BaseLBTC.sol#L108-L119

https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/blob/45ee91f30c14011662326b05d9663d589189e529/smart-contracts/contracts/LBTC/BridgeTokenAdapter.sol#L211-L223

Recommendation

Remove the `config.limit == 0` half from `RateLimitsV2.setRateLimit()`.

Discussion

hashxtree

We want to kept as-is for future callers of the library. Improved docs.
https://github.com/lombard-finance/smart-contracts/pull/517

Bauchibred

Acknowledged.

Issue L-6: Unused `IERC20` / `SafeERC20` import and `using` directive in `StakedLBTC` [RESOLVED]

Source: https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/issues/12

Summary

`StakedLBTC` imports `IERC20` / `SafeERC20` and declares `using SafeERC20 for IERC20`, none of which are used.

Vulnerability Detail

No `SafeERC20.` call or `IERC20` usage exists in `StakedLBTC.sol`; the import and `using` directive are dead.

Impact

Informational.

Code Snippet

https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/blob/main/smart-contracts/contracts/LBTC/StakedLBTC.sol#L19

Tool Used

Manual Review

Recommendation

Remove the unused imports and `using` directive.

Discussion

hashxtree

https://github.com/lombard-finance/smart-contracts/pull/518

defsec

Fix is confirmed

Issue L-7: Proof-mint rate-limit bucket is shared across all users (`address(this)` keying) on NativeLBTC & BridgeTokenAdapter [RESOLVED]

Source: https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/issues/13

Summary

All permissionless proof-mint paths on `NativeLBTC` and `BridgeTokenAdapter` consume a **single** rate-limit bucket keyed by `address(this)`, rather than a per-recipient or per-caller bucket. As a result, unrelated users' proof mints compete for one shared per-window quota: heavy (but fully legitimate) proof-mint activity from one user can temporarily exhaust the bucket and cause other users' unrelated proof mints to revert with `RateLimitExceeded` until the bucket decays.

Vulnerability Detail

Both contracts route every proof mint through `_consumeMinterLimit(address(this), …)`:

471:473:smart-contracts/contracts/LBTC/NativeLBTC.sol

```
// Actually mint (contract-level bucket; caller is not `MINTER_ROLE`)
_consumeMinterLimit(address(this), amountToMint);
_mint(recipient, amountToMint);
```

412:414:smart-contracts/contracts/LBTC/BridgeTokenAdapter.sol

```
// Actually mint (contract-level bucket; caller is not `MINTER_ROLE`)
_consumeMinterLimit(address(this), action.amount);
$.bridgeToken.mint(
```

The shared bucket is `minterRateLimits[address(this)]`, configured via `setMinterLimit(address(this), …)`. On `NativeLBTC` it is shared by `mintV1`, `batchMintV1`, and the `CLAIMER_ROLE` `mintV1WithFee` / `batchMintV1WithFee` paths (the fee leg uses `_transfer`, so only the deposit `amount` is charged, but every proof flow still draws from this one quota). On `BridgeTokenAdapter` it is shared by `mintV1` / `batchMintV1`.

Scenario

1. `address(this)` bucket configured on `NativeLBTC` with `limit = L`, `window = W`.
2. Alice submits valid `mintV1` proofs for her own deposits totalling ⩽ `L` within the window.

3. Bob's unrelated, fully valid `mintV1` reverts `RateLimitExceeded` until the bucket decays – even though Bob's deposit is independent and consortium-approved.

The same contention exists between heavy `CLAIMER` `mintV1WithFee` traffic and public `mintV1`.

Impact

Temporary, cross-user denial of service on the permissionless proof-mint path: one user's activity can delay other users' unrelated, legitimate proof mints within the same window.

Code Snippet

471:473:smart-contracts/contracts/LBTC/NativeLBTC.sol

412:414:smart-contracts/contracts/LBTC/BridgeTokenAdapter.sol

Tool Used

Manual Review

Recommendation

Decide explicitly whether the proof-mint cap is intended to be **protocol-wide** or **per-user**:

- If per-user fairness is desired, key the proof-mint bucket per recipient (`minterRateLimits[recipient]`), or use separate buckets for the permissionless path vs the `CLAIMER` fee path.
- If a single protocol-wide cap is intended, document clearly (in NatSpec and operator runbooks) that `address(this)` is one global quota shared by all users' proof mints, so the limit `L` is sized for aggregate throughput and short-window contention is expected.

Discussion

hashxtree

https://github.com/lombard-finance/smart-contracts/pull/519

Issue L-8: BridgeTokenAdapter `changeBridgeToken` should be removed [RESOLVED]

Source: https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/issues/14

Summary

`BridgeTokenAdapter.changeBridgeToken()` remains callable even though it should only be for migration testing and removed

Vulnerability Details

`BridgeTokenAdapter.changeBridgeToken()` is still callable despite the documentation to be removed. The function forwards directly into _changeBridgeToken(), which swaps the stored `bridgeToken` address without any migration, pause, or state-consistency checks.

Code snippet

```
function changeBridgeToken(
    address newVal
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    _changeBridgeToken(newVal);
}

function _changeBridgeToken(address newValue) internal {
    Assert.zeroAddress(newValue);
    BridgeTokenAdapterStorage storage $ = _getBridgeTokenAdapterStorage();
    address prevValue = address($.bridgeToken);
    $.bridgeToken = IBridgeToken(newValue);
    emit BridgeTokenChanged(prevValue, newValue);
}
```

Impact

Code for testing is still available in live production which can cause for a switch of the adapter onto a different underlying token.

Recommended Mitigation Steps

Remove `changeBridgeToken()` from production code.

Discussion

hashxtree

https://github.com/lombard-finance/smart-contracts/pull/520

Bauchibred

Fix confirmed

Issue L-9: Stale natspec in BridgeTokenAdapter's `_confirmDeposit` & `_changeBridgeToken` [RESOLVED]

Source: https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/issues/15

Summary

Some stale natspec are still present in production.

Vulnerability Details

`BridgeTokenAdapter._confirmDeposit()` currently documents @param $ NativeLBTC storage whereas it takes `BridgeTokenAdapterStorage`.

Also, _changeBridgeToken() is still preceded by the same `treasury not zero` comment as is used for `_changeTreasury()`, even though `_changeBridgeToken()` does not change the treasury.

Impact

Informational.

Code snippets

https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/blob/45ee91f30c14011662326b05d9663d589189e529/smart-contracts/contracts/LBTC/BridgeTokenAdapter.sol#L467-L475

https://github.com/sherlock-audit/2026-06-lombard-mint-rate-limits-v2-june-25th-2026/blob/45ee91f30c14011662326b05d9663d589189e529/smart-contracts/contracts/LBTC/BridgeTokenAdapter.sol#L505-L511

Recommendation

Update the stale comments.

Discussion

hashxtree

https://github.com/lombard-finance/smart-contracts/pull/521

Bauchibred

Fix confirmed

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.